

# Evidence Crosses, Authority Does Not: Receiver-Owned Admission of Agent Tool Calls Within and Across Organizations

The Chio Project

<https://github.com/bb-connor/chio>

September 2026

## Abstract

An agent calling a tool through an ordinary client holds whatever authority its host process holds; across an organization boundary the receiver has only the sender's word. Our protocol rests on one rule: authority is minted and verified only inside the kernel that dispatches, against roots it activated itself, and nothing in a request can add a root; only signed, content-addressed evidence crosses. What crosses is the receipt, a kernel-signed record of one admission decision named by its content hash. Two co-signed statements name receipts. The inbound one is evidence a request presents about a receipt the receiver already holds; resolving it before dispatch is where the security property lives. The outbound one the receiver mints over its own new receipt once that receipt is durable, and the peer signs those bytes unparsed; it gates the response, never the dispatch. Neither is a bearer credential: a statement names only what the receiver stores. Hardened with every check a competent operator would write, a composition of a pinned peer identity, a signing policy engine, and a replay table reaches receiver locality, single use, and audience binding, but not admission binding: most of what the receiver binds has no carrier there. The contribution is the carrier, not the check.

## 1 The rule

A buyer agent at one company asks a vendor agent at a second to stage a refund against the vendor's payment ledger. The vendor's software must decide, before touching that ledger, whether to obey. An API key and a transport signature establish who is calling and nothing about whether this call, with these arguments, under the agreement the two companies signed, was already authorized, already performed, or rests on a delegation revoked four minutes ago. Afterward each holds a log about itself that no third party can reconcile. Having the sender prove more makes the receiver's security a function of the sender's correctness; a shared ledger or broker makes that substrate a participant in every call.

The rule here avoids both. **Authority is minted and verified only inside the kernel that dispatches, against roots that kernel activated itself, and nothing carried in a request can add a root. Only signed, content-addressed evidence crosses an organization boundary.** A signature proves who signed some bytes; it cannot decide whether the receiver should act on them, because that depends on facts only the receiver holds: which keys it pinned, which agreement it activated and when, whether this continuation has been spent.

That is a placement argument, composing with the stack, not replacing it [16]. Alignment training, transcript monitors, and the sender's policy engine change how often the kernel says no; none knows the callee's budget is exhausted or this argument hash already dispatched. The receiver decides.

One object carries it: the receipt, a kernel-signed record of one admission decision named by its own content hash. Two co-signed statements name receipts. The **inbound** statement is evidence a request presents about a receipt the receiver already holds, signed by two keys it pinned beforehand; the pre-dispatch hook resolves it against the receiver's own stores, and the security property lives there. The **outbound** statement the receiver mints over its own new receipt, once that receipt is durable, and the peer signs those bytes unparsed; it gates the response, never the dispatch. Neither is a bearer credential: what either names the receiver stores. Agent protocols define how a tool is called [2, 1]; none defines the record of what was admitted. We call the mediating component the kernel: Chio is userspace, not an operating-system kernel.

A pinned peer identity, a signing policy engine, and a replay table, wired with every check a competent operator would write, reach three of the four properties of Section 6: receiver locality, single use, and audience binding. Admission binding they do not reach, for a missing carrier rather than a forgotten check: most of what a receiver binds has no field in the composed request. The contribution is the carrier, not the check.

## 2 Threat model and trusted base

The kernel sits between an agent and the tool servers whose transport it owns: child processes on their standard streams, HTTP tools it fronts, sidecars it proxies. It does not confine the agent, whose model, orchestrator, prompt, and libraries are untrusted; one that opens its own socket reaches whatever that socket reaches.

The adversary is given everything outside the receiving kernel. It controls every request field this paper names; it controls the peer kernel and can make it sign anything with the peer’s key; it controls the network and may drop, delay, reorder, duplicate, and replay; and it may present any receipt, statement, lease, or governance record it has observed, in any combination. It does not hold the receiving kernel’s signing key and cannot write to its stores. The question is what it can cause a receiving kernel to dispatch.

The trusted base is the receiving kernel’s process, its signing key, and its stores; everything outside is verified or assumed. The assumptions appear once, in Table 1.

Table 1: Assumptions outside the receiving kernel, by the identifiers of the implementation’s assumption registry. The registry has 16 entries; the ten this protocol rests on are listed.

| Identifier                             | What is assumed                                                                                                                                                                                     |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASSUME-ED25519                         | Ed25519 meets existential unforgeability under chosen-message attack for keys the kernel trusts.                                                                                                    |
| ASSUME-SHA256                          | SHA-256 is collision resistant over receipt and statement preimages.                                                                                                                                |
| ASSUME-CANONICAL-JSON                  | The production canonical encoder agrees byte for byte with the mechanized one on strings, bounded integers, finite arrays, and objects ordered by UTF-16 code unit. Floats are outside that domain. |
| ASSUME-SQLITE-ATOMICITY                | A single-row insert commits atomically, so a primary-key conflict is a reliable witness that a row existed. Cross-row recovery and ordering are not assumed.                                        |
| ASSUME-OS-CLOCK                        | The injected clock reports Unix time within the operator’s declared tolerance.                                                                                                                      |
| ASSUME-NETWORK-TRANSPORT, ASSUME-TLS   | Delivery is not assumed reliable; arriving messages are assumed not rewritten below the signature checks, and remote surfaces authenticate their endpoints.                                         |
| ASSUME-SUBPROCESS-ISOLATION            | Effects after an allow, and isolation between tool processes, rest on operating-system boundaries.                                                                                                  |
| ASSUME-FEDERATED-ORIGIN-CLASSIFICATION | The receiving edge marks cross-organization origin from a transport-authenticated peer identity; a request misclassified as local never reaches these checks.                                       |
| ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND | Revocation liveness only: recurring delivery under weak fairness, bounded skew, operator-bounded partition healing. Safety does not depend on it.                                                   |

One assumption is deliberately absent: that the two signing keys on a co-signed statement belong to different organizations. One operator holding both satisfies every check here; key independence is a deployment property the protocol carries no evidence of. Nor does the peer’s countersignature constrain this adversary: the receiver authors an outbound statement’s bytes and the peer signs them unparsed, so it buys a record a non-adversarial peer cannot disown.

## 3 The receipt

A receipt is a JSON object with a fixed field set, encoded canonically under RFC 8785 [15]. Three of its projections are hashed, and they nest:  $b$  drops the `id`, the `signature`, and the two optional fields recording how it was signed; the canonical receipt body  $\beta$  is  $b$  plus the `id`;  $r$  is the whole receipt as it travels. Write  $D$  for SHA-256 of the canonical encoding and  $k$  for the kernel key published in the body; signatures are Ed25519 [12].

$$id = D(b), \quad \sigma = \text{Ed25519}_k(\text{JCS}(\{\text{body}: b, \text{id}: id\}))$$

Each projection has its own wire field:  $id$  names the receipt,  $D(\beta)$  is the subject digest of a co-signed statement, inbound or outbound, and  $D(r)$  is a binding reference’s `remote_receipt_sha256`. No two agree, because  $\beta$  adds the `id` to  $b$  and  $r$  adds  $\sigma$  to  $\beta$ . The signature covers a fourth preimage, the object above, which the optional BBS signature joins as a third key, so that field is covered by  $\sigma$  and not by  $id$ . A receipt renamed is a receipt unsigned. Three properties follow.

**Recompute and refuse.** Where the path holds the evaluated content the signer is handed the content, not a digest: it recomputes the digest inside the trust boundary and refuses to sign if the body’s claimed digest differs.

**Decisions, not successes.** A denial is a receipt of the same shape, built, signed, and persisted on the same path as an allow. Three fields fix what kind of statement a receipt is, so a weaker one cannot pose as a

stronger one: `receipt_kind` separates a mediated decision from a passive trace, `boundary_class` says whether the kernel could have prevented the effect or only observed it, and `trust_level` records the evidence class. A body claiming a decision on a kind that cannot make one is not signed; a message rejected before it had a decision to record returns its code and no receipt.

**Permission, proof, and price in one object.** When the exercised grant carries monetary caps the hold taken before dispatch is reconciled into the receipt’s financial block: `cost_charged`, `currency`, `budget_remaining`, `budget_total`, the delegation depth, and the root budget holder. The receipt an auditor reads to learn the call was allowed is the object a biller reads to learn what it cost. Settlement between organizations is out of scope, and that is our whole claim about commerce.

The vendor kernel’s receipt for the refund step, abridged at the metadata:

```
{ "id": "d342a59d...6fba2e3f", "timestamp": 1766000001,
  "capability_id": "lease-vendor-c-refund", "tool_server": "vendor-c.payments",
  "tool_name": "stage_refund",
  "action": { "parameters": { "caseRef": "refund-250", "tool": "stage_refund",
    "workflowId": "wf-chio-refund-001" },
    "parameter_hash": "47e6e096...d5a6464b" },
  "decision": { "verdict": "allow" }, "trust_level": "mediated",
  "receipt_kind": "mediated_decision", "boundary_class": "prevent",
  "tool_origin": "caller_executed", "redaction_mode": "none",
  "content_hash": "c9abf68a...d9803987",
  "policy_hash": "82cfeae3...fb2ca3fd",
  "metadata": { "attribution": { "delegation_depth": 0, "grant_index": 0, ... },
    "chio_runtime": { "accepted": true, "destructive": true,
      "lease_id": "lease-vendor-c-refund", "step_index": 2,
      "reserved_treaty_continuation_id": "continuation:runtime-loopback:2" }, ... },
  "kernel_key": "31debe55...cfa498bc",
  "signature": "a9e4762e...1ba28505" }
```

Checked on these bytes:  $b$  hashes to `d342a59d...6fba2e3f`, the object’s id;  $\beta$  to `31118bb6...4781b3f9`, the subject digest of the statement the vendor mints over it after persisting it (Section 5);  $r$  to `63f1f35a...8c93e3e8`, that statement’s `remote_receipt_sha256`. The signing preimage over  $b$  verifies under the embedded kernel key. This receipt carries no financial block because the exercised grant carries no cap.

## 4 Admission

Admission is the one operation. Every tool call, local or foreign, goes through it, and every path that reached a decision produces a signed receipt.

1. Verify the presented capability against the kernel’s own trusted issuer set: signature, time window, subject binding, and the revocation state of the leaf and every ancestor; resolve the grants matching this tool and these arguments.
2. Open a durable admission operation, a fenced write-ahead record whose state machine has 18 states. Its fence is a store lease with an epoch: no second coordinator can advance it.
3. Run the guard pipeline. Deny dominates, and so does error: a guard that produces no verdict denies.
4. Run the admission hook, where Section 5 runs for a request classified as foreign; one carrying no agreement context is denied here.
5. Take the budget hold, charging the reservation before dispatch.
6. Revalidate the mutable inputs, mark the operation dispatch-committed, and only then hand the call to the tool server over a transport the kernel owns.
7. Run output guards, build the receipt, recheck its fields against the admitted inputs, sign and persist it, then return.

Three orderings are the whole of its value. **Durable before dispatch:** no dispatch occurs without a committed pre-dispatch record. Before that commit recovery releases the hold; after it, with no outcome known, recovery terminates the operation as outcome-unknown rather than inventing one. An output not matching the committed digest terminates as denied-after-delivery, with a signed deny and nothing captured against the hold.

**Receipt before response:** the receipt is persisted before the caller sees the result, and a kernel that cannot persist a receipt denies. The log is a write-ahead record of what was admitted, not a trace of what was observed.

**Consume after compare:** step 4 spends a foreign request’s single-use continuation after every comparison against the inbound statement it presented, and after both signatures on that statement verify, and before the local admission report that rebuilds this call’s binding from the live request. A statement the receiver refuses cannot burn a continuation a legitimate call still needs. A statement that passes every comparison on

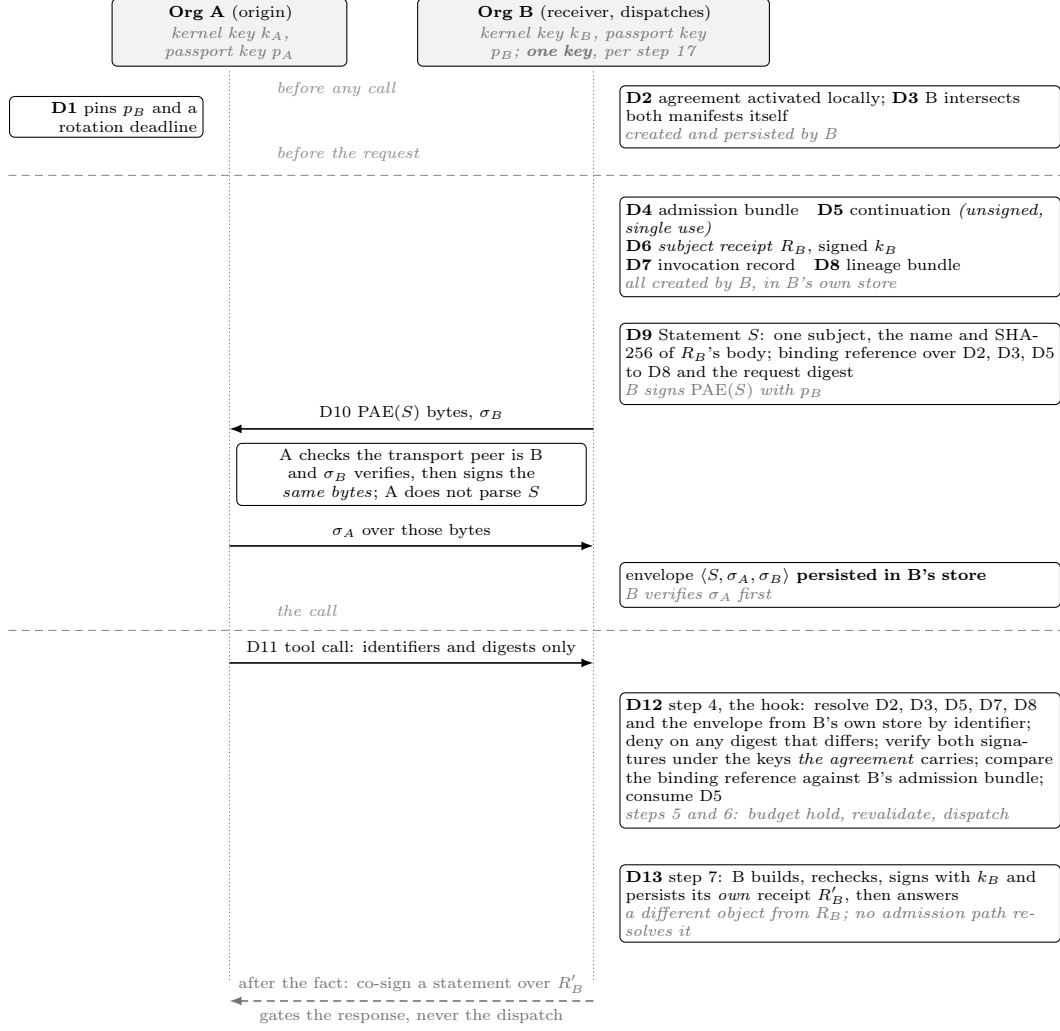


Figure 1: One cross-organization call, end to end. Every object the receiving kernel checks at step 4 was created by the receiver and was already in the receiver's own store before the request arrived; the request carries identifiers and digests and nothing else. The statement the hook verifies (D9, D10) has as its subject a receipt the *receiver* signed (D6), not the peer's, and that is a different receipt from the one the receiver writes for this call at step 7 (D13). The dashed return is the outbound co-signature: the reference kernel mints it on every federated request and fails the response if it cannot, but it is minted after the dispatch it describes and no admission path resolves it.

a request the report then denies does spend one, and gets it back through a release scoped to that admission; an unconfirmed release is recorded as ambiguous rather than released.

## 5 Cross-organization admission

A call from another organization is the same admission operation, and both co-signed statements occur in it. The inbound one is named by the request and resolved by the pre-dispatch hook at step 4; it adds no authority, because the receiver's issuer set, pinned peers, agreements, and clocks are what they were before the request arrived. Section 6 concerns it, and the outbound one closes this section. Figure 1 follows one call end to end.

### 5.1 Receiver-owned resolution

Two organizations pin each other out of band, each recording the other's kernel identity, the passport key it co-signs with, and a rotation deadline. The agreement is an activated record naming exactly two kernels,

the action classes in scope, a validity window, and a revocation epoch. Each participant publishes a signed manifest, the action classes it accepts and at what intensity on the ascending ladder **observation**, **guarded**, **receipt\_backed**, **partition\_contingency**, **maintenance**, and each receiver activates the agreement and both manifests locally and intersects them per action class itself: the higher-intensity mode wins, the stricter co-signing requirement wins, and a destructive class below **receipt\_backed** rejects. A pin the receiver has not activated is a denial.

The request may carry identifiers. It may not carry trust. Its agreement context is names and digests, each a lookup key into a store the receiver owns; Figure 2 is the whole of what such a name reaches. Eleven further key names are refused wherever they appear, seven naming a trust root, bundle, agreement record, manifest, signing key or peer directory and four naming dynamic trust input or peer discovery, with **request\_smuggled\_trust\_root** and **request\_smuggled\_dynamic\_trust**.

## 5.2 The co-signed statement

One wire form serves both directions: an in-toto Statement v1 [21] in a DSSE envelope [18] whose predicate is **snake\_case** and rejects unknown fields. Its single subject names an invocation identifier and carries the SHA-256 of that receipt's canonical body of Section 3. Below is the refund envelope, the outbound one the vendor minted over that receipt, then its payload. An inbound subject instead names a receipt the receiver already held, in the reference path one record per receiver process named identically on every call, so it carries no per-call information and the per-call binding rests on the binding reference alone.

```
{ "payloadType": "application/vnd.in-toto+json",
  "payload": "eyJfdHlwZSI6Imh0dHBzOiI8vaW4tdG90by5pby9T...",
  "signatures": [
    { "keyid": "fdf72a088f18f7399e8c52bce448441501f759a595a86980e5d9a422a01e5d55",
      "sig": "3QZ09JP4YLFmvgzwy5PWqYTWFCCQQkUo/DR07YyFB8pD+bA6iBGwyRTtI+1zzI7A..." },
    { "keyid": "e37abcd79efd08db94518a903952ffe9f8af7e8b4684b63591631cef8e8871c8",
      "sig": "R01i8tcBzYcEPAPRQqNsbCUowFH0hzspjr5Mm98mFu33+XcT0fDxdWf48btYzmd..." } ] }
```

```
{ "_type": "https://in-toto.io/Statement/v1",
  "predicateType": "chio.bilateral-cosign-invocation.v1",
  "subject": [ { "name": "chio-receipt:d342a59d...6fba2e3f",
    "digest": { "sha256": "31118bb6...4781b3f9" } } ],
  "predicate": {
    "invocation_id": "d342a59d...6fba2e3f", "tool_name": "stage_refund",
    "tool_server_a": { "kernel_id": "did:chio:buyer-kernel", "alg": "ed25519",
      "passport_key_fingerprint": "fdf72a08...a01e5d55" },
    "tool_server_b": { "kernel_id": "did:chio:vendor-c", "alg": "ed25519",
      "passport_key_fingerprint": "e37abcd7...8e8871c8" },
    "co_sign": "bilateral_required", "consistency_model": "totally-ordered",
    "consistency_anchor": "chio:consistency:wf-chio-refund-001:2",
    "cross_org_visibility": "federated", "timestamp_unix_ms": 176600001000,
    "tool_args_hash": { "alg": "sha256", "value": "47e6e096...d5a6464b" },
    "capability_lease_ref": { "lease_id": "lease-vendor-c-refund",
      "issuer": "did:chio:buyer-kernel", "expires_at_unix_ms": 1766000030000,
      "scope_digest": { "alg": "sha256", "value": "262768fe...b9f835aa" } },
    "governance_receipt_ref": { "receipt_id": "gov-refund-stage-authorization",
      "kernel_id": "did:chio:buyer-governance",
      "digest": { "alg": "sha256", "value": "21143a93...f161e3c0" } },
    "policy_evaluation_summary": { "joint_disposition": "allow",
      "server_a_verdict": { "verdict": "allow", "policy_id": "buyer-policy:stage_refund", ... },
      "server_b_verdict": { "verdict": "allow", ... } },
    "treaty_binding_ref": {
      "treaty_id": "treaty:runtime-loopback:2",
      "action_class_id": "workflow.cross_kernel.stage_refund",
      "consistency_model": "totally-ordered",
      "treaty_scope_sha256": "0ff230e9...d1f2b21c",
      "ladder_intersection_sha256": "93dcc7cf...af79d3d1",
      "admission_report_sha256": "91d74509...1f646522",
      "continuation_sha256": "8ef2f348...366b93f4",
      "lineage_bundle_sha256": "28d816a3...d3b504ba",
      "request_sha256": "47e6e096...d5a6464b",
      "outcome_sha256": "c9abf68a...d9803987",
      "local_receipt_sha256": "9cbe4438...ad017121",
      "remote_receipt_sha256": "63f1f35a...8c93e3e8",
      "lease_refs": [ "lease-vendor-c-refund" ],
      "governance_refs": [ "gov-refund-stage-authorization" ],
      "signer_kernel_ids": [ "did:chio:buyer-kernel", "did:chio:vendor-c" ] } }
```

Both signatures cover the DSSE pre-authentication encoding of the payload, which here begins **DSSEv1 28 application/vnd.in-toto+json**, and each **keyid** is the SHA-256 of its raw public key. Recomputed from the committed bytes, **request\_sha256** is **tool\_args\_hash.value** and the receipt's **action.parameter\_hash**.

## 5.3 The verifier, in the order it runs

Table 2 is the shipped order; two components decide, in two disjoint code families. The conforming verifier returns the specification's dotted codes, a closed set of 24 strings of which it emits 16 and the envelope layer the rest. The kernel's pre-dispatch hook answers from the kernel's runtime failure-code set of 505 strings, 24 of them agreement-scoped, and maps every envelope-layer failure to **chio\_treaty\_unverified\_required\_evidence**.

Table 2: The conforming verifier, 26 steps, grouped by where the state each step reads comes from. Every failure returns one code from the closed set; the specification maps each step to its own. Steps 8 to 14 are the envelope layer, called with the two keys the receiver pinned.

| #     | Check                                                                                                                                                                                                                                                                              |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1–5   | Media type, at least one signature, payload decodes and parses; predicate type <b>chio.bilateral-cosign-invocation.v1</b> ; Statement v1; exactly one subject                                                                                                                      |
| 6–7   | Both kernel identifiers are in the pinned peer set and each pin’s fingerprint equals the declared one                                                                                                                                                                              |
| 8–13  | Exactly two signatures; re-canonizing reproduces the payload bytes; predicate schema and strict rules; both verdicts present, equal, and in {allow, deny}; subject name is <b>chio-receipt:</b> plus the invocation identifier; the two pinned keys and their identifiers distinct |
| 14    | <i>Only now</i> : compute the pre-authentication encoding, select each signature by a pinned key’s identifier, and verify both                                                                                                                                                     |
| 15–16 | Each pinned peer’s manifest reference is present and fresh, and the peer is active at the pinned epoch height                                                                                                                                                                      |
| 17–19 | Resolve the receipt by invocation identifier in the receiver’s own store; its signature verifies, its tool name and argument hash agree, its kernel key is the pinned second-party key, and subject name and digest match the resolved body                                        |
| 20–22 | Both verdicts resolve and agree; the lease resolves with matching issuer, expiry and scope digest and is unexpired; the tool name is in the action-class table; a receipt-backed class resolves its governance record                                                              |
| 23–26 | Binding cross-checks (Table 3), consistency model and anchor, co-signing mode; return the verified material                                                                                                                                                                        |

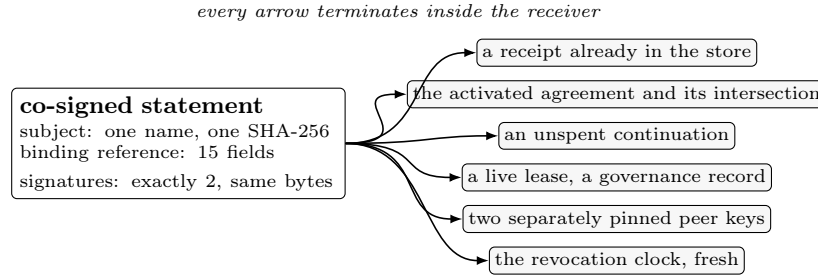


Figure 2: The mechanism. The statement carries no authority of its own: its subject, the binding fields the receiver compares, and each of its two keys resolve only against records the receiving kernel placed in its own stores before the request arrived.

Table 3 and Section 8’s denial experiment report the hook, which decides live calls.

Signatures are verified last and the key lookup first, the opposite of the usual advice: neither decider can verify until it knows which keys to verify under, neither takes them from the envelope, and no gate before step 14 consults a secret, so the ordering costs nothing in soundness. They read different receiver-owned records for those keys, the verifier its pin set and the hook the activated agreement, and either can be ahead of the other, so a rotation carried into one and not the other is admitted by whichever decider holds the key the envelope names. Steps 17 to 19 are the verifier’s alone: neither the hook nor the kernel’s treaty verification resolves a receipt by invocation identifier or compares the subject digest, so on a live call the receipt binding rests on `local_receipt_sha256` and `remote_receipt_sha256` against the lineage bundle and the invocation record.

## 5.4 Consume, bind, co-sign

A replayed statement is validly signed, so no signature defeats replay. A single-use continuation does, and the receiver mints it before the call: a record binding the parent receipt and session anchor, the capability, the action class, the target tool, a nonce, the two kernel identities, and a validity window, whose digest is one of the 15 bound fields. It carries no signature and needs none: what authenticates it is residency in the receiver’s own store and the digest the request presents, which sharpens the rule rather than relaxing it. Two operations run on one table keyed by that identifier. *Consume* inserts, ignoring a conflict, and reads the affected row count; zero rows means the continuation was already spent and the call is denied with `chio_treaty_continuation_replay`. *Release* deletes the row for that identifier and that admission, and runs only on a later pre-dispatch denial, so a rejected call does not burn a continuation. Once dispatch commits the row stays: at-most-once, on purpose.

The receiver then builds its own admission report and binds the accepted material into the receipt it signs, checking only that the statement's `admission_report_sha256` is well-formed hex before overwriting it with its own report's digest.

The outbound statement is minted here, a different object: its subject is the receipt this call just produced, the receiver authors its pre-authentication bytes, and the peer signs them without parsing. The local receipt is persisted first, so a refusal or a transport failure is recorded against a receipt that already exists. The reference kernel mints it on every federated request and fails the response when it cannot, and no admission path resolves it. Its second signature constrains nothing the receiver decided, since the Section 2 adversary holds the peer's key: evidence, not prevention.

## 6 Security

Four properties carry the protocol, each stated over the adversary of Section 2 and reduced to an assumption in Table 1. All four are about the inbound statement, the evidence a request presents about a receipt the receiver already holds; the outbound one gates the response, never the dispatch.

**Admission binding.** *If a receiving kernel dispatches under cross-organization admission, then both keys the activated agreement pins signed the pre-authentication bytes of the statement  $S$  the request presented; every identifier and digest the request presented resolved, in the receiver's own stores, to an artifact whose digest the binding reference of  $S$  names; every other leaf of  $S$  the receiver compares equals the value it computed for this request; two are bound to a receiver-held set instead, the lease expiry to the instants strictly after the receiver's clock and the lease issuer to the agreement's two participants; and the continuation  $S$  names was unconsumed when admission began.* Suppose not. Each equality is byte equality between a digest the receiver computed and one the statement carries, so a difference that passed means two distinct objects share a SHA-256 image or a canonical encoding, contradicting ASSUME-SHA256 or the injectivity of the encoder under ASSUME-CANONICAL-JSON. A difference in a signature contradicts ASSUME-ED25519, and a continuation consumed twice contradicts ASSUME-SQLITE-ATOMICITY. What remains is completeness of the comparison set, which is the leaf enumeration of Section 7, not an assumption.

The property is over the hook, which decides live calls: its receipt binding runs through the lineage bundle and the invocation record, not the statement's subject, and a deployment that wants the subject digest checked before dispatch runs the conforming verifier of Section 5 too.

**Receiver locality.** *The roots, pins, agreements, action-class tables, and registries used to decide a request are a function of the receiver's state before the request arrived, and are independent of the request's content.* Extraction reads a fixed set of key names and ignores the rest, and what it reads are identifiers, each the argument of a total lookup over receiver-owned state; the refused names of Section 5 make smuggled trust fail loudly.

**Single use.** *For a continuation identifier  $c$ , at most one admission ever reaches dispatch with  $c$  bound to it.* Under ASSUME-SQLITE-ATOMICITY exactly one of any set of concurrent inserts on a primary key creates the row; every other denies. Release deletes only the row keyed by both  $c$  and the admission that took it, and runs only before dispatch, on the ordering of Section 4. And  $c$  is minted by the receiver before the request and named by  $S$ , so the receiver, not the signer, decides how many authorizations exist.

**Audience binding.** *A statement accepted by receiver  $R$  is not accepted by a different receiver  $R'$  unless  $R'$  activated an agreement of the same identifier and content, resolved the same intersection, and holds the same continuation unconsumed.* The receiver's identity enters through the continuation rather than the statement's name for it: the continuation  $R'$  resolved names  $R'$  as its target and both its kernels as participants of the agreement  $R'$  activated, which has exactly two, so  $R'$  admits only as the second of them, under the key that agreement pins.

**Substitution over the binding reference.** Table 3 is an executed corpus over the 15 binding fields, evidence PS-T12. Each case re-signs the substituted statement with both participant keys, so both signatures verify over the bytes the hook decides on. Of the 22 cases one substitutes nothing and admits, so every denial is attributable to the field that changed, and every denial leaves the dispatch counter at zero. There `binding_mismatch` and `unverified_evidence` abbreviate `chio_treaty_dsse_binding_mismatch` and `chio_treaty_unverified_required_evidence`.

Table 3: Substituting one binding field at a time with a well-formed different value, both signatures valid over the substituted statement, as decided by the pre-dispatch hook. Ten fields are compared on every cross-organization admission, the signer list twice, as a set and in order. Four more are compared only when the admission resolves the artifact they bind, which the action class’s required-evidence set decides; the refund call resolves both the invocation record and the lineage bundle. The fifteenth is never compared, and the case that substitutes it is admitted.

| Field                                     | Receiver-side value it must equal                                                                                                                                           | First code                                            | Compared when    |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|------------------|
| <code>treaty_id</code>                    | the agreement the receiver resolved for this request                                                                                                                        | <code>binding_mismatch</code>                         | always           |
| <code>treaty_scope_sha256</code>          | the digest the receiver computes over its own agreement record                                                                                                              | <code>binding_mismatch</code>                         | always           |
| <code>ladder_intersection_sha256</code>   | the intersection digest the receiver resolved                                                                                                                               | <code>binding_mismatch</code>                         | always           |
| <code>admission_report_sha256</code>      | <i>nothing</i> : 64 hex characters, then overwritten by the receiver’s own report digest                                                                                    | none: the substituted call is admitted and dispatches | never            |
| <code>continuation_sha256</code>          | the resolved continuation’s digest                                                                                                                                          | <code>binding_mismatch</code>                         | always           |
| <code>action_class_id</code>              | the action class the receiver resolved for this request                                                                                                                     | <code>binding_mismatch</code>                         | always           |
| <code>consistency_model</code>            | the model the resolved intersection fixed                                                                                                                                   | <code>binding_mismatch</code>                         | always           |
| <code>request_sha256</code>               | this dispatch’s argument hash, and the predicate’s own <code>tool_args_hash</code>                                                                                          | <code>binding_mismatch</code>                         | always           |
| <code>lease_refs</code>                   | the lease in the receiver’s own admission bundle                                                                                                                            | <code>binding_mismatch</code>                         | always           |
| <code>governance_refs</code>              | the governance record in that bundle                                                                                                                                        | <code>binding_mismatch</code>                         | always           |
| <code>signer_kernel_ids</code> , as a set | exactly two, equal as a set to the agreement’s participants                                                                                                                 | <code>binding_mismatch</code>                         | always           |
| <code>signer_kernel_ids</code> , in order | the invocation record’s order; a reversal resolves the two pinned keys transposed, and strict envelope verification refuses that pairing before the ordered comparison runs | <code>unverified_evidence</code>                      | always           |
| <code>lineage_bundle_sha256</code>        | the lineage bundle the receiver resolved                                                                                                                                    | <code>binding_mismatch</code>                         | lineage required |
| <code>outcome_sha256</code>               | the outcome digest in the receiver’s invocation record                                                                                                                      | <code>binding_mismatch</code>                         | record required  |
| <code>local_receipt_sha256</code>         | the lineage bundle’s root receipt digest, and the invocation record’s                                                                                                       | <code>binding_mismatch</code>                         | either required  |
| <code>remote_receipt_sha256</code>        | the lineage bundle’s leaf receipt digest, and the invocation record’s                                                                                                       | <code>binding_mismatch</code>                         | either required  |

The whole-statement attack, another validly co-signed statement from the same pair of kernels, is stopped by the continuation rather than by any digest: it names one the receiver either does not know or has already consumed.

The report digest is not the only leaf compared against nothing the receiver holds. The others are the statement’s subject digest and timestamp, its consistency anchor, both parties’ policy identifiers and versions, and the governance record’s kernel identifier, digest, and digest algorithm. Each is still held to a declared shape, and the qualification is part of the claim: substituting one with a value of its declared shape moves no decision, while a non-hexadecimal report digest and an empty policy identifier are denied. The residual is additive as well: the optional halves of the lease scope digest and the two policy rationale codes are permitted by the strict validator, never validated and never read, so an adversary holding both keys adds them and the receiver admits.

The window in which a statement can be presented is the intersection of the lease, governance, and continuation intervals the receiver itself resolved, each revocable and re-scoped per counterparty between calls rather than one constant applied to every authorization, and after the first dispatch it is zero.

**Revocation.** Revocation is the only part of this protocol that depends on something arriving. Each origin publishes a signed root carrying a monotone epoch counter and an issue time, and a kernel installs a snapshot only if its epoch strictly exceeds the installed one, by compare-and-swap, so a replayed frame cannot take effect and a concurrent install cannot be lost; it then applies its own freshness window to the issue time before trusting it. Safety holds unconditionally, in every partition, because both halves are local predicates that never wait for a message: no call is allowed against a revocation the kernel has observed, and a call evaluated against a stale snapshot is denied.

Liveness holds only under ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND, and it bounds the time before a revocation is observed, not the number of evaluations, which is unbounded. Let  $F$  be the freshness window,  $T$  the origin’s control-poll period,  $L$  the latency of one admitted call, and  $k$  the number of calls made



before one is decided against the newer snapshot. A still-connected receiver denies no later than  $T + kL$  after publication, and a cut receiver no later than  $F + T + kL$ , because its snapshot must first age past  $F$ . With  $F = 500$  ms,  $T = 250$  ms,  $k = 1$ , and the median  $L$  of Section 8, those are 456.8 ms connected and 956.8 ms cut.

## 7 What is mechanized

The decision core is a pure function: verify a capability, resolve grants, evaluate, sign. It is modeled in Lean 4 [8], whose published theorem inventory carries 196 declarations and one explicit axiom, a collision-resistance idealization registered against ASSUME-SHA256. No serializer, protocol step, or kernel behavior is axiomatized; an unproved property is an assumption-registry entry.

Two results bear on this paper. Occurrence theorems make the predicate language fail closed: an atom unsupported, or undefined for the admission view, anywhere in a predicate forces denial on every view, by induction. The second is the comparison set of Section 6: a classification of every leaf of the co-signed predicate is total by exhaustiveness over the leaf type, and modelled acceptance is a statement gate conjoined with the named comparisons, so acceptance pins every equality-bound leaf, puts the two set-bound leaves in their receiver-held sets, and survives replacing an uncomparing leaf with a value of its declared shape.

Completeness is cross-checked, not transcribed: the executed leaf corpus derives its enumeration from the wire type, reads the Lean module, and fails if the leaf paths or the classifications disagree.

**Lean does not verify the running Rust.** These theorems are about the modelled predicate and the classification; the corpus is what links them to the shipped hook, on one fixture. There are four kinds of tripwire, each failing loudly when the two drift apart: named Rust functions mirrored to Lean definitions by content hash and checked on every change; 20 production functions extracted through a Rust-to-Lean pipeline with equivalence theorems that must rebuild byte-identically; 47 bounded model-checking harnesses over public entry points, one rejecting an untrusted issuer before dispatch; and the cross-language encoder corpus of Section 8.

We report survivors, not percentages. Of 45 mutants in the Lean development 33 were killed and 12 survived, each with an open issue, and of 166 in the Rust proof-facing sources 160 were killed, 5 were unviable, and 1 survived. A survivor is a place where a theorem is weaker than it looks.

## 8 Evaluation

Everything below ran on one machine: 12 Neoverse-N1 cores, 46.9 GiB, Linux 6.17.0-1020-oracle aarch64, compiler 1.94.1 at the pinned toolchain. Percentiles interpolate between order statistics; every interval is a percentile bootstrap interval for the median beside it.

**Two kernels in separate processes.** Two kernels were brought up as separate operating-system processes, each with its own keys, stores, issuers, agreements, leases, and receipt log, with a third process driving calls over iroh QUIC, relays disabled; the topology record reports 1 host. The co-signing and revocation hops are the shipped lanes unchanged; the request hop uses a transport defined for this experiment, since no shipped lane carries a tool call. Admitted calls: 100 calls, p50 206.803 ms, p99 234.520 ms at the driver, each costing 3 co-signing connections, counted by the receiver. Of that round trip the receiver’s evaluation window is 203.787 ms at the median and the driver’s evidence preparation 3.297 ms.

Denied calls: 7 scenarios driven over the wire, 140 calls, p50 58.821 ms, p99 88.462 ms. They return `chio_treaty_missing_scope`, `chio_treaty_scope_hash_mismatch`, `chio_treaty_missing_intersection`, `chio_treaty_intersection_mismatch`, `chio_treaty_missing_required_evidence`, `request_smuggled_trust_root`, and `request_smuggled_dynamic_trust`. In every one the receiver’s dispatch counter was zero: no call reached a tool.

Across the run 88 further calls were denied because the receiver’s snapshot had aged past its 500 ms freshness window while the origin publishes every 250 ms: 0 of the 100 admitted attempts, 3 in the denial experiment, 85 in the revocation experiment. That is the safety rule of Section 6 firing on the clock, not on a revocation; they were retried and excluded.

Revoking at the origin and measuring to the receiver’s first denial took 322.899 ms at the median over 10 repeats, interval 178.977 to 368.288 ms, a median of 2 calls. With the link cut the receiver serves until its snapshot leaves the freshness window and then denies, 744.681 ms at the median over 10 repeats, interval 609.423 to 916.110 ms, a median of 2 calls, counting a denial only once it persists for three further calls. The

Table 4: Local cost decomposition. The first two rows are complete kernel paths measured per invocation with both kernels in one process; the rest are components.

| Path or component                            | p50         | p99 or max   | samples |
|----------------------------------------------|-------------|--------------|---------|
| Admitted call, both kernels in one process   | 13.863 ms   | 26.176 ms    | 200     |
| Refused before dispatch                      | 7.073 ms    | 18.646 ms    | 400     |
| Receipt append to the store                  | 4681.643 us | 13566.644 us | 500     |
| Receipt signing                              | 229.340 us  | 270.205 us   | 100     |
| Receipt verification                         | 337.470 us  | 344.582 us   | 100     |
| Inbound two-signature statement verification | 339.127 us  | 350.904 us   | 100     |
| Cross-boundary admission decision            | 20.688 us   | 21.207 us    | 100     |

Table 5: A receiver composed from 4 parts an operator already has, driven through the corpora of this paper under two wirings. The parts are a tool-call server, a peer identity pinned out of band, a policy engine that evaluates a local rule set and signs its decision, and a receiver-side replay table. *Composed* wires each part the way it documents itself; *hardened* adds every check a competent operator would write, among them a denylist over the context names the receiver owns and a replay table keyed by the identifier inside the signed bytes rather than one the caller picks. No part requires either check, and nothing in the composition records which one an operator wrote. Admissions are over the 22 attacks the corpus drives, as 20 distinct experiments. Chio’s own record is at most 15.5 times heavier per call, a bound rather than a measurement, since the two stores are read by two procedures.

| Property of Section 6  | Composed        | Hardened |
|------------------------|-----------------|----------|
| Admission binding      | fails           | fails    |
| Receiver locality      | fails           | holds    |
| Single use             | holds, weakened | holds    |
| Audience binding       | holds           | holds    |
| Attacks admitted       | 9               | 2        |
| Durable bytes per call | 763.9           | 866.3    |

partitioned receiver stops rather than falling back to its last good answer. On 3 of 4 revoke records and 0 of 10 cut records the pre-dispatch release could not be confirmed and was recorded as ambiguous, the path Section 4 names; no call dispatched.

**What admission costs locally.** The shape of Table 4 is the result: the cryptography is not the expense. Appending the receipt durably costs 13.8 times what verifying the inbound statement costs, and 226 times the receiver’s decision over already resolved evidence, where the comparisons of Table 3 run. Durability dominates, which is the price of Section 4’s ordering. The refusal row is a lower bound: its fixture refuses on a unanimous deny verdict, an early exit reaching neither those comparisons nor the signature checks. Held at saturation for 60 s the same path admitted 3543 calls at 59.0 calls per second with 0 denials and 40.1 MiB of receipt-store growth.

The negative corpus is 20 in-process cases, each asserting its expected code: 8 through the dispatch-capable hook with the tool-invocation counter asserted zero, 12 against the verifier directly, alongside 20 bilateral and 50 generic replay cases.

**What a composition of existing parts reaches.** Table 5 is the comparison, and hardening is decisive: it closes 7 attacks and carries 3 of the four properties. What no wiring reaches is admission binding, and the reason is a missing carrier rather than a missing check: 10 of the 16 substitution rows, covering 9 of the 15 binding fields, have no carrier in the composed request at all, so there is nothing to compare. The witness is an unmodified admissible call presented against a receiver whose own record for that agreement has advanced a version: the receiver holds a version, the request pins none, and no check compares a fact the request cannot carry. Hardening is not free either: the last row prices the second durable write it makes.

**Verification by someone who was not there.** The 50.6 KiB evidence package for the refund workflow verifies with no access to either kernel, every receipt, statement, lease, and governance record, in 49.7 ms at the median over 100 runs. A frozen corpus of 14 receipt cases holds four independent implementations to the

same canonical body bytes and verdict.

**Reproduction.** Each run writes its commit, environment, raw samples, and the macro file this paper compiles from, so a printed number and a retained one cannot drift. The comparison writes a duration only from a run on a clean input tree and an unloaded host, and otherwise records that it did not: its corpora and verdicts are above, its timings are not.

## 9 Limitations and related work

The cross-organization run of Section 8 put the two kernels in separate processes on one host, not on two separately administered machines. That removes the shared address space, store, and key material, which are what the claim is about; it leaves the shared operating system and clock source. Two machines would remove those and change the latencies. The rule does not depend on that experiment; the strength of the demonstration does.

Section 8’s alternative is one construction judged on one corpus; only the missing carrier is a fact about the request format rather than the wiring. Continuation consumption is at-most-once, so a crash between consume and dispatch loses the call rather than repeating it: right for a refund, an avoidable retry for an idempotent read. The chain closes in the kernel for one hop, since the next continuation in a longer chain is minted outside it. Denial paths below the verifier still return a rendered detail alongside the code; separating them is unfinished. Receipts are batched into Merkle checkpoints in the RFC 6962 shape [14] with offline inclusion proofs, and that is the whole claim: local audit evidence with verifiable inclusion, not a public append-only log, and with no anti-equivocation mechanism, so a kernel signing two contradictory checkpoints is detectable only by someone holding both.

**Related work.** Capability discipline without ambient authority is the tradition this protocol sits in [17, 23], saying exactly where a proof stops is seL4’s discipline [13], and attenuable credentials are old and good: the trust-management line [9, 5], macaroons [3], and the modern token families [22, 6], which carry caveats without a kernel, a decision record, or a price. Kerberos [20] crossed the boundary by transferring authority, workload identity by exchanging trust bundles and minting a local credential for a foreign workload [19], and token exchange by converting one issuer’s token into another’s [11]. All three move authority across the boundary, which is the step this protocol declines to take. Interchain light clients [10] solve pinned keys plus a monotone clock for chains rather than tool calls, and policy engines decide from a request they are handed [7]. The envelope shapes we reuse [18, 21], transparent registration [4], and certificate transparency [14] answer how an artifact was built, who may register a statement, and how to publish verifiable records. What none of them defines is the object that is at once the authorization decision, the audit record, the accounting entry, and the evidence another organization’s kernel verifies against state it alone holds.

**What we believe.** The rule is smaller than the system that implements it, and we think it is the part that lasts. The receiver has to decide, from facts only it holds, and the record of that decision has to outlive the connection and be checkable by someone who trusts neither side.

## References

- [1] Agent2Agent Project. A2A: Agent2agent protocol specification. Specification, 2025.
- [2] Anthropic. Model context protocol specification. Specification, 2024.
- [3] Arnar Birgisson et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [4] Henk Birkholz et al. An architecture for trustworthy and transparent digital supply chains. IETF SCITT working group, Internet-Draft, 2025.
- [5] Matt Blaze et al. The KeyNote trust-management system version 2. RFC 2704, 1999.
- [6] Geoffroy Couprie and Clément Delafargue. Biscuit: A decentralized authorization token. Specification, v3, 2023.
- [7] Joseph W. Cutler et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2024.
- [8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, 2021.
- [9] Carl M. Ellison et al. SPKI certificate theory. RFC 2693, 1999.
- [10] Christopher Goes. The interblockchain communication protocol: An overview. arXiv:2006.15918, 2020.
- [11] Michael B. Jones et al. OAuth 2.0 token exchange. RFC 8693, 2020.
- [12] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017.
- [13] Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
- [14] Ben Laurie et al. Certificate transparency. RFC 6962, IETF, 2013.
- [15] Anders Rundgren et al. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.
- [16] Jerome H. Saltzer et al. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
- [17] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

- [18] Secure Systems Lab. DSSE: Dead simple signing envelope. Secure Systems Lab specification, v1.0, 2021.
- [19] SPIFFE Project. SPIFFE: Secure production identity framework for everyone. Specification, federation and trust bundles, 2024.
- [20] Jennifer G. Steiner et al. Kerberos: An authentication service for open network systems. In *USENIX Winter Conference*, 1988.
- [21] Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [22] UCAN Working Group. UCAN: User controlled authorization networks. Specification, v1.0, 2025.
- [23] Robert N. M. Watson et al. Capsicum: Practical capabilities for UNIX. In *USENIX Security Symposium*, 2010.